1.

We are given n items (numbered 1 to n).
Initially, each item's buy count = 0.

We have 3 types of queries:

1 x → Person buys all items from 1 to x (inclusive).
So for all items i where $1 \le i \le x$, increase buy[i]++.

2 x → Person buys all items from x to n (inclusive).
So for all items i where $x \le i \le n$, increase buy[i]++.

3 x → Query: return the item number $\ge x$ that has been bought the most times.

If multiple items have the same maximum count, return the one with minimum difference from x (i.e., closest to x).

2.

Problem Statement

You have a cache of fixed size n.

Two types of queries:

1 key -1 → Get the value at key. If not present, return -1.
Also, if present, increment the frequency of usage of that key.

2 key value → Insert (key, value) into the cache.

If key already exists → update the value and increase frequency.

If cache is full → evict the least frequently used (LFU) key.

If tie (multiple keys with same frequency), evict the least recently used (LRU)*